

Lab 4: Coding on Modern Processors or Say Exploiting Instruction-Level Parallelism

Hung-Wei Tseng

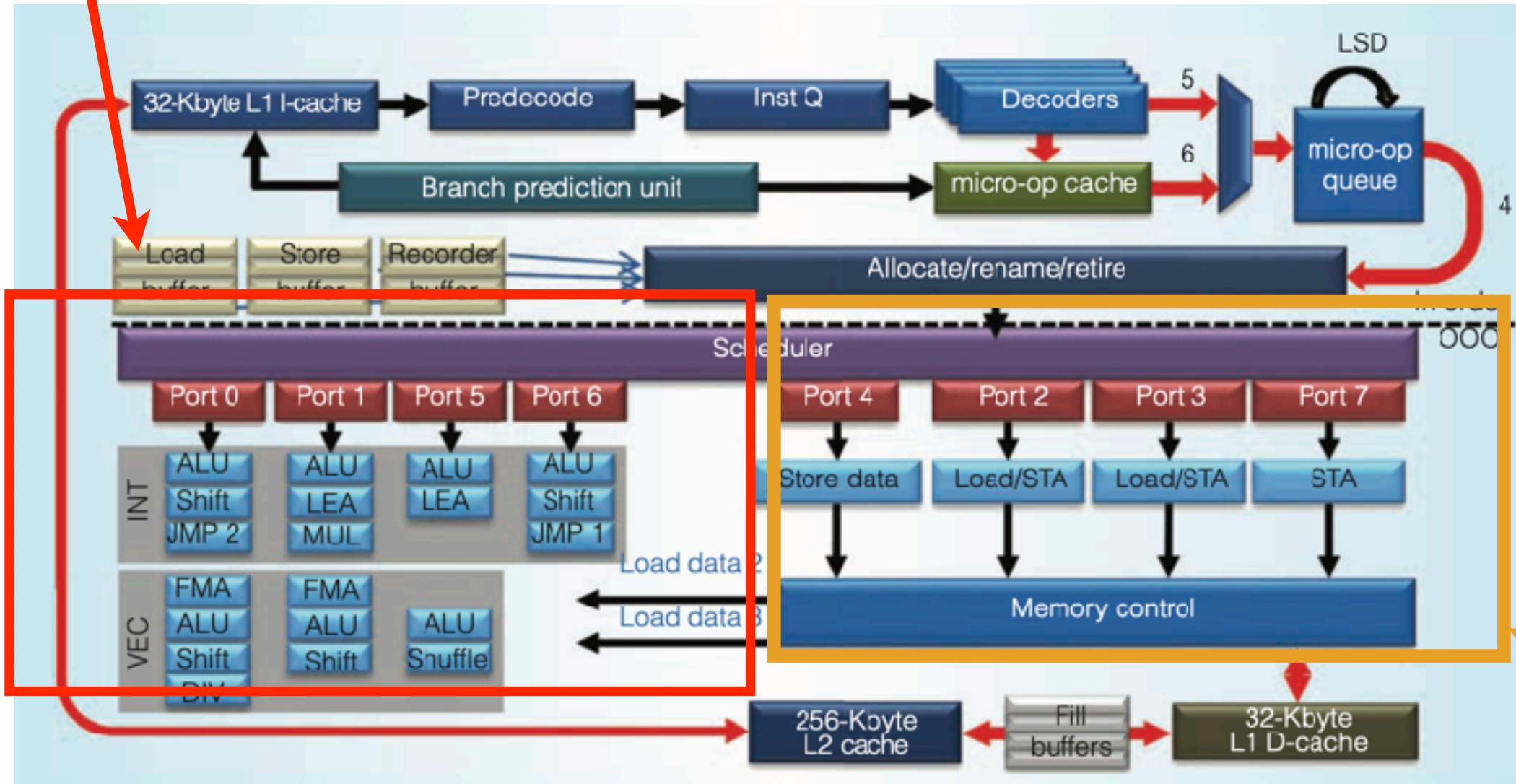
Parallelism in modern computers

- Instruction-level parallelism
 - The ability to execute multiple instructions concurrently
- Thread-level parallelism
 - The ability to execute multiple program instances concurrently
- Data-level parallelism
 - The ability to process data concurrently

Instruction-level parallelism

Q9: Intel Sky Lake

4-issue ALU pipeline



$$\begin{aligned} \text{MinCPI} &= \frac{1}{8} \\ \text{MinINTInst.CPI} &= \frac{1}{4} \\ \text{MinMEMInst.CPI} &= \frac{1}{4} \\ \text{MinBRInst.CPI} &= \frac{1}{2} \end{aligned}$$

4-issue memory pipeline

Compiler optimizations

Q1: Strength Reduction — Use a Simpler Operation

- Instead of multiplications and divisions, how can we compute?

a * 43			a / 43			a % 43		
leaq	(%rdi,%rdi,4), %rax		movabsq	\$428994048225803526, %rdx		movabsq	\$428994048225803526, %rdx	
leaq	(%rdi,%rax,4), %rax		movq	%rdi, %rax		movq	%rdi, %rax	
leaq	(%rdi,%rax,2), %rax		imulq	%rdx		imulq	%rdx	
			sarq	\$63, %rdi		movq	%rdi, %rcx	
			movq	%rdx, %rax		sarq	\$63, %rcx	
			subq	%rdi, %rax		movq	%rdx, %rax	
						subq	%rcx, %rax	
						leaq	(%rax,%rax,4), %rdx	
						leaq	(%rax,%rdx,4), %rdx	
						leaq	(%rax,%rdx,2), %rdx	
						movq	%rdi, %rax	
						subq	%rdx, %rax	

– form 2: **imulq s**

- one operand is %rax
- The other operand given in the instruction
- product is stored in %rdx (high-order part) and %rax (low order part)
→ full 128-bit result

sarq k, Dest Dest = Dest $\gg$ k (arithmetic)

%rdi = a
%rax = %rdi + 4*%rdi (= 5a)
%rax = %rdi + 4*%rax (=21a)
%rax = %rdi + 2*%rax (=43a)

%rdi = a
%rdx = %rdi * 428994048225803526
 (= 428994048225803526a)
%rdi = %rdi >> 63 (take a's signed bit)
%rax = 428994048225803526a's higher-order
part only — = 428994048225803526a >> 64 (= a/43.00000000065)
%rax = %rax - %rdi

Good reference for instruction latencies

<https://uops.info/>

Let's start with a program

```
void swap(int* a, int* b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

// A function to implement bubble sort
void bubbleSort(int *arr, int n)
{
    int i, j;
    for (i = 0; i < n - 1; i++)
        // Last i elements are already
        // in place
        for (j = 0; j < n - 1 - i; j++)
            if (arr[j] > arr[j + 1])
                swap(&arr[j], &arr[j + 1]);
}

// Function to print an array
void printArray(int *arr, int size)
{
    int i;
    for (i = 0; i < size; i++)
        printf("%d\t", arr[i]);
}
```

```
int main()
{
    unsigned N = 131072;
    int *data;
    if(argc > 1)
        N = atoi(argv[1]);
    data = (int *)malloc(sizeof(int)*N);

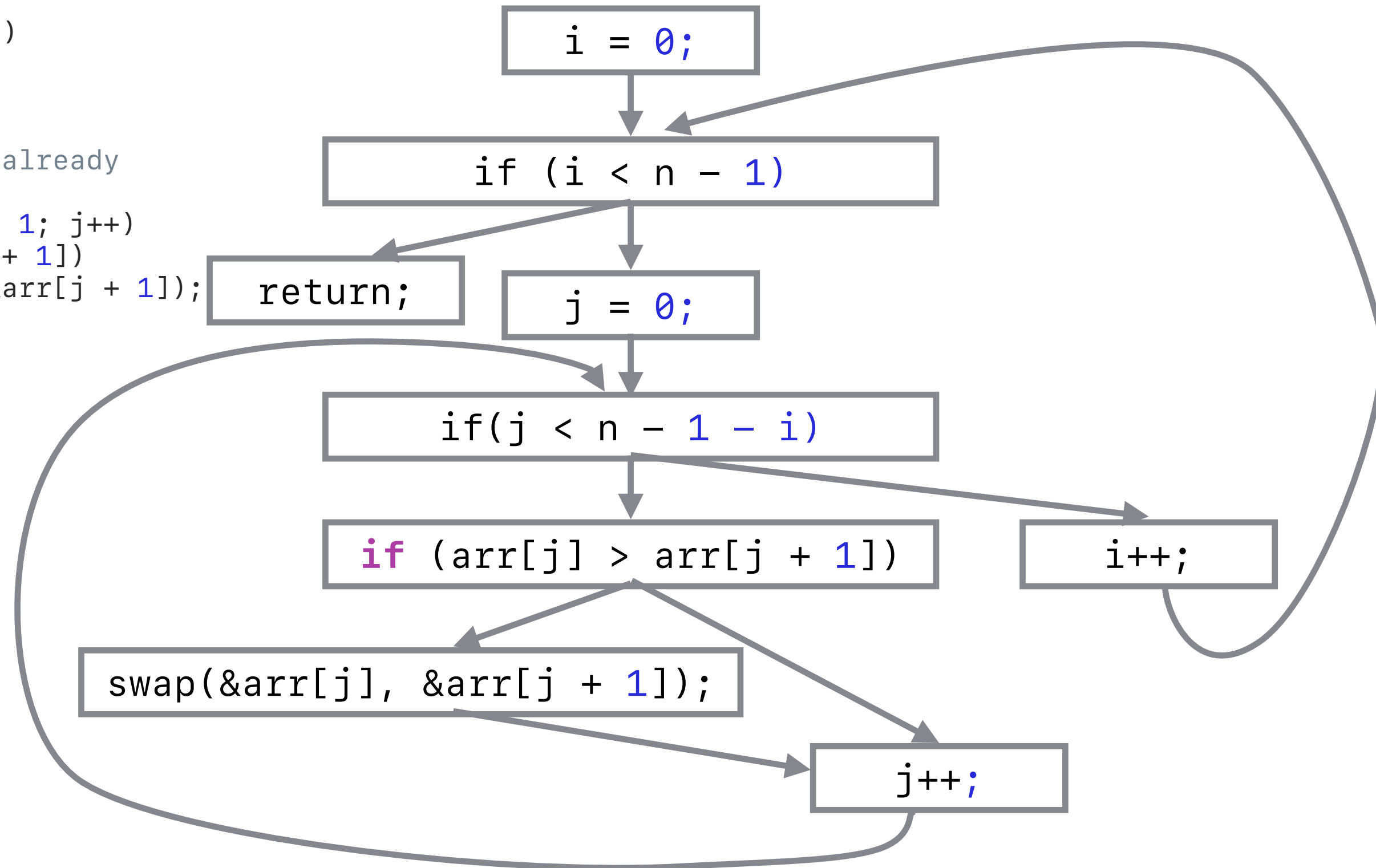
    for (unsigned i = 0; i < N; ++i)
        data[i] = rand();
    bubbleSort(data, N);
    printArray(data, N);
    return 0;
}
```


Control flow graph

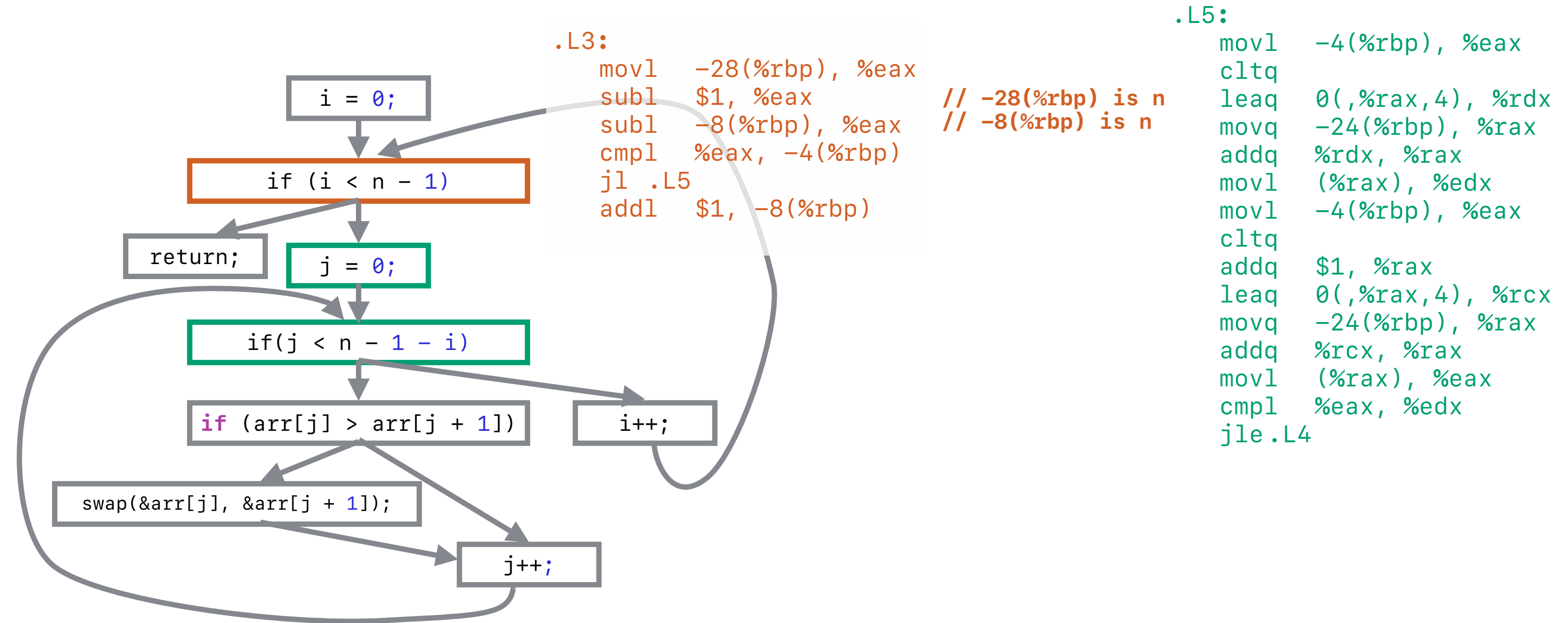
- A graph shows all possible route of executing a piece code
- A "directed" graph consists of **basic blocks**
- A **basic block** is a sequence of instructions that will always execute together
 - A sequence of instructions until we reach a "branch"
 - Compiler typically can aggressively reorder instructions within a basic block

Control flow graph

```
void bubbleSort(int *arr, int n)
{
    int i, j;
    for (i = 0; i < n - 1; i++)
        // Last i elements are already
        // in place
        for (j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(&arr[j], &arr[j + 1]);
}
```



Control flow graph and code



Where do you see opportunities of compiler optimizations?

```
void swap(int* a, int* b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Should become register variables

// A function to implement bubble sort

```
void bubbleSort(int *arr, int n)
{
    int i, j;
    for (i = 0; i < n - 1; i++)
        // Last i elements are already
        // in place
        for (j = 0; j < n - 1 - i; j++)
            if (arr[j] > arr[j + 1])
                swap(&arr[j], &arr[j + 1]);
}
```

Common sub-expression
elimination (CSE)

// Function to print an array

```
void printArray(int *arr, int size)
{
    int i;
    for (i = 0; i < size; i++)
        printf("%d\t", arr[i]);
}
```

```
int main()
{
```

```
    unsigned N = 131072;
```

```
    int *data;
```

```
    if (argc > 1)
```

```
        N = atoi(argv[1]);
```

```
    data = (int *)malloc(sizeof(int)*N);
```

```
    for (unsigned i = 0; i < N; ++i)
```

```
        data[i] = rand();
```

```
    bubbleSort(data, N);
```

```
    printArray(data, N);
```

```
    return 0;
```

```
}
```

The unoptimized swap function and its caller stub

```
movq    -24(%rbp), %rax
addq    %rcx, %rax
movq    %rdx, %rsi
movq    %rax, %rdi
call    swap
```

put base address of a & b
in %rdi and %rsi

RSP (stack pointer) register points
to top of stack

Instruction	Effective Operations
pushq src	subq \$8, %rsp movq src, (%rsp)
popq dest	movq (%rsp), dest addq \$8, %rsp

```
swap: save %rbp in the stack and obtain the
.LFB6: current stack pointer
        endbr64
        pushq %rbp move base addresses into the stack
        movq %rsp, %rbp
        movq %rdi, -24(%rbp)
        movq %rsi, -32(%rbp)
        movq -24(%rbp), %rax
        movl (%rax), %eax
        movl %eax, -4(%rbp)
        movq -32(%rbp), %rax
        movl (%rax), %edx
        movq -24(%rbp), %rax
        movl %edx, (%rax)
        movq -32(%rbp), %rax
        movl -4(%rbp), %edx
        movl %edx, (%rax)
        nop
        popq %rbp
        ret
```

swap values completely on memory
operations...

Where do you see opportunities of compiler optimizations?

function inlining

```
void swap(int* a, int* b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Should become register variables

// A function to implement bubble sort

```
void bubbleSort(int *arr, int n)
{
```

```
    int i, j;
```

```
    for (i = 0; i < n - 1; i++)
```

```
        // Last i elements are already
```

```
        // in place
```

```
        for (j = 0; j < n - 1 - i; j++)
```

```
            if (arr[i] > arr[j + 1])
```

```
                swap(&arr[j], &arr[j + 1]);
```

```
    // Function to print an array
```

```
void printArray(int *arr, int size)
```

```
{
```

```
    int i;
```

```
    for (i = 0; i < size; i++)
```

```
        printf("%d\t", arr[i]);
```

```
}
```

Common sub-expression
elimination (CSE)

```
int main()
{
```

```
    unsigned N = 131072;
```

```
    int *data;
```

```
    if (argc > 1)
```

```
        N = atoi(argv[1]);
```

```
    data = (int *)malloc(sizeof(int)*N);
```

```
    for (unsigned i = 0; i < N; ++i)
```

```
        data[i] = rand();
```

```
    bubbleSort(data, N);
```

```
    printArray(data, N);
```

```
    return 0;
```

```
}
```


Optimization with -O1

- Some variables are now in registers
- Leading to fewer memory accesses
- Load Effective Address (lea) — is not a memory instruction, but an arithmetic instruction compute the “address” only
 - `leal -1(%rsi), %eax ==
xorl %eax, %eax
addl %rsi, %eax
sub $1, %eax`
- Strength Reduction — use a simpler operation

Q6, Q7, Q8: function call overhead

Before

```
movq    %rbx, %rsi
call    swap
jmp     .L5
```

put base address of a
in %rsi

RSP (stack pointer) register points
to top of stack

Instruction	Effective Operations
pushq src	subq \$8, %rsp movq src, (%rsp)
popq dest	movq (%rsp), dest addq \$8, %rsp

```
swap:
    movl    (%rdi), %eax
    movl    (%rsi), %edx
    movl    %edx, (%rdi)
    movl    %eax, (%rsi)
    ret

optimized swap operations
```

2 instructions in caller, 5
instructions in callee = 7

After

```
.L4:
    addq    $4, %rax
    cmpq    %r8, %rax
    je      .L10
.L5:
    movl    (%rax), %edx
    movl    4(%rax), %ecx
    cmpl    %ecx, %edx
    jle     .L4
    movl    %ecx, (%rax)
    movl    %edx, 4(%rax)
    jmp     .L4
```

Only two lines optimized swap
operations
Only 2!

The compiler can leverage the
fact that they're a[j] and a[j+1]

-02

- Loop invariant code motion
- Constant propagation
- Function Inlining
- More you can find when you do your assignment 4!

Q3 and Q4 — similar to this demo!

```
for (unsigned i = 0; i < 100000; ++i) {  
    int threshold = std::rand();  
    for (unsigned i = 0; i < arraySize; ++i) {  
        if (data[i] >= threshold) // Branch X  
            sum ++;  
    }  
}
```

	If data is not sorted?	If data is sorted?
The prediction accuracy of X before threshold	50%	100%
The prediction accuracy of X after threshold	50%	100%

Q5: Loop unrolling — how does that reduce “branch” instructions

```
for (i=0;i<size;i++)
```

```
    sum += a[i];
```

```
① movslq    (%rdi), %rdx
```

```
② addq    $4, %rdi
```

```
③ addq    %rdx, %rax
```

```
④ cmpq    %rcx, %rdi
```

```
⑤ jne     .L21
```

```
⑥ movslq    (%rdi), %rdx
```

```
⑦ addq    $4, %rdi
```

```
⑧ addq    %rdx, %rax
```

```
⑨ cmpq    %rcx, %rdi
```

```
⑩ jne     .L21
```

```
⑪ movslq    (%rdi), %rdx
```

```
⑫ addq    $4, %rdi
```

```
⑬ addq    %rdx, %rax
```

```
⑭ cmpq    %rcx, %rdi
```

```
⑮ jne     .L21
```

```
⑯ movslq    (%rdi), %rdx
```

```
⑰ addq    $4, %rdi
```

```
⑱ addq    %rdx, %rax
```

```
⑲ cmpq    %rcx, %rdi
```

```
⑳ jne     .L21
```

20 instructions
and 4 branches,

4 iterations

If we have n iterations?

$5 \times n$ instructions

n branches

If we have n iterations?

$19 \times \frac{n}{8}$ instructions

$\frac{n}{8}$ branches

19 instructions
and 1 branch
for 8 iterations,

```
new_size = size >> 3
```

```
for (i=0;i<new_size*8;i++)
```

```
    sum += a[i];
```

```
① .L26: movslq    (%rdi), %r9
```

```
②      movslq    8(%rdi), %r10
```

```
③      addq    $32, %rdi
```

```
④      movslq    -20(%rdi), %r11
```

```
⑤      movslq    -16(%rdi), %rax
```

```
⑥      addq    %rcx, %r9
```

```
⑦      movslq    -28(%rdi), %rcx
```

```
⑧      movslq    -12(%rdi), %rsi
```

```
⑨      movslq    -8(%rdi), %r8
```

```
⑩      addq    %rcx, %r9
```

```
⑪      movslq    -4(%rdi), %rcx
```

```
⑫      addq    %r10, %r9
```

```
⑬      addq    %r11, %r9
```

```
⑭      addq    %rax, %r9
```

```
⑮      addq    %rsi, %r9
```

```
⑯      addq    %r8, %r9
```

```
⑰      addq    %r9, %rcx
```

```
⑱      cmpq    %rdx, %rdi
```

```
⑲      jne     .L26
```

```
⑳ .L26: movslq    (%rdi), %r9
```

```
㉑      movslq    8(%rdi), %r10
```

```
㉒      addq    $32, %rdi
```

```
㉓      movslq    -20(%rdi), %r11
```

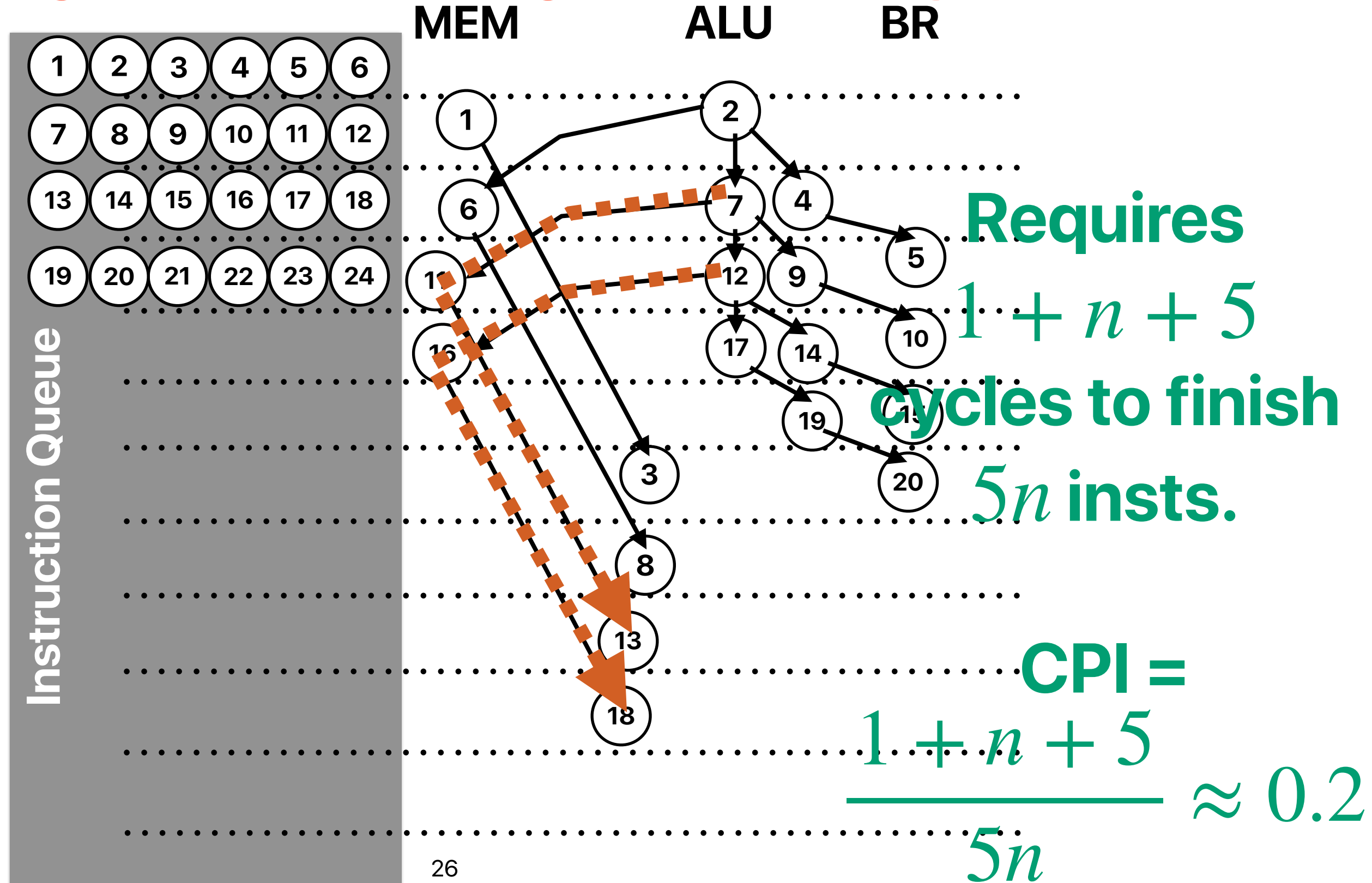
```
㉔      movslq    -16(%rdi), %rax
```

Q11—Q15, Q17: critical path of programs

- Critical path: the sequence of instructions determines the execution time
 - If we remove this instruction from the program and the execution time will change, it's on the critical path
 - Otherwise, it's not on the critical path
- How to find critical path
 - Draw the dataflow/data dependency graph!
 - The length of each edge should represent the latency of the operation
 - Identify the path that defines the total latency

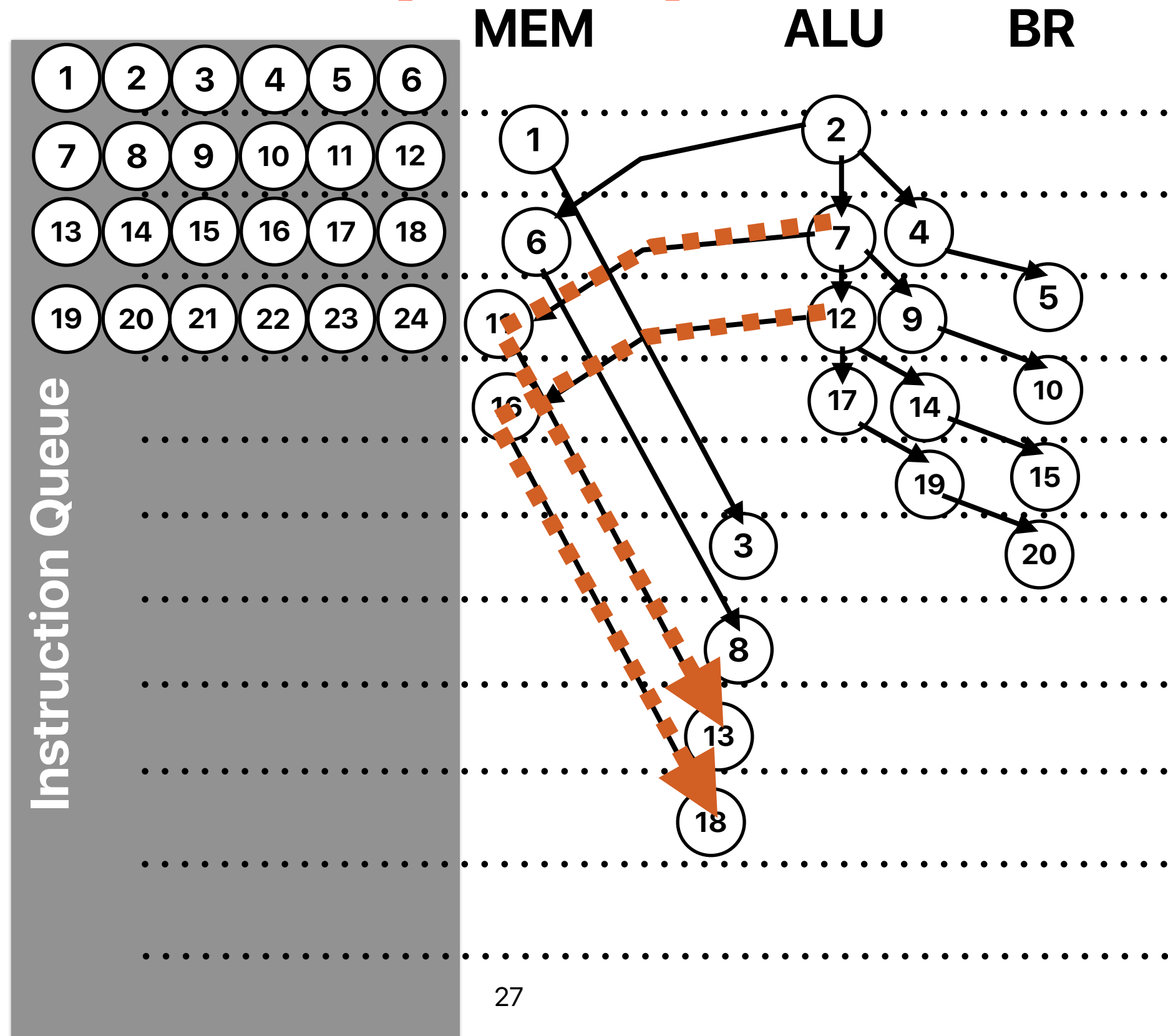
Through data flow graph analysis

```
for (i=0;i<size;i++)  
    sum += a[i];  
① movl (%rdi), %ecx  
② addq $8, %rdi  
③ addq %rcx, %rax  
④ cmpq %rdx, %rdi  
⑤ jne .L3  
⑥ movl (%rdi), %ecx  
⑦ addq $8, %rdi  
⑧ addq %rcx, %rax  
⑨ cmpq %rdx, %rdi  
⑩ jne .L3  
⑪ movl (%rdi), %ecx  
⑫ addq $8, %rdi  
⑬ addq %rcx, %rax  
⑭ cmpq %rdx, %rdi  
⑮ jne .L3  
⑯ movl (%rdi), %ecx  
⑰ addq $8, %rdi  
⑱ addq %rcx, %rax  
⑲ cmpq %rdx, %rdi  
⑳ inc .L3
```



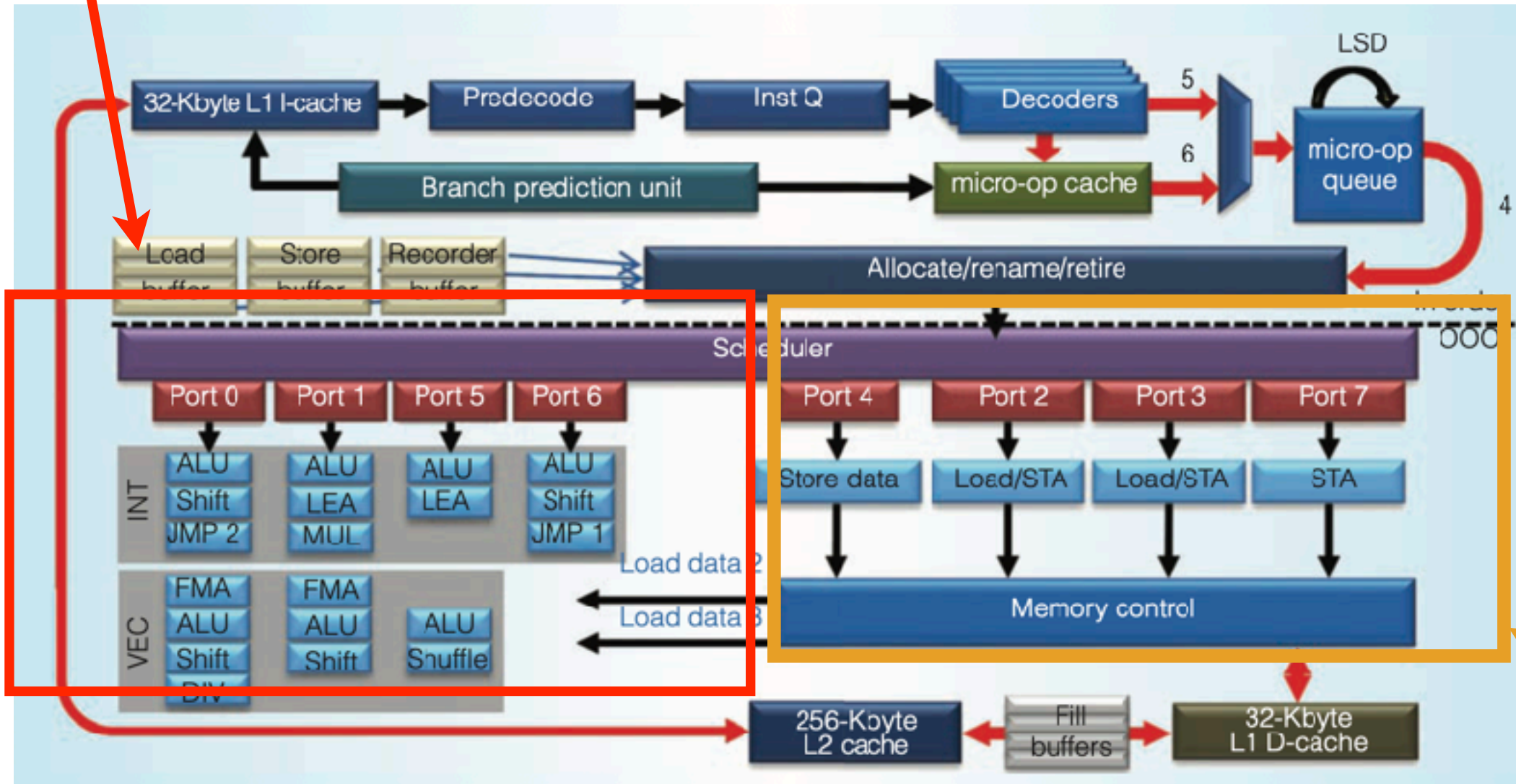
How can we improve performance?

```
for (i=0;i<size;i++)  
    sum += a[i];  
① movl (%rdi), %ecx  
② addq $8, %rdi  
③ addq %rcx, %rax  
④ cmpq %rdx, %rdi  
⑤ jne .L3  
⑥ movl (%rdi), %ecx  
⑦ addq $8, %rdi  
⑧ addq %rcx, %rax  
⑨ cmpq %rdx, %rdi  
⑩ jne .L3  
⑪ movl (%rdi), %ecx  
⑫ addq $8, %rdi  
⑬ addq %rcx, %rax  
⑭ cmpq %rdx, %rdi  
⑮ jne .L3  
⑯ movl (%rdi), %ecx  
⑰ addq $8, %rdi  
⑱ addq %rcx, %rax  
⑲ cmpq %rdx, %rdi  
⑳ inc     .L3
```



Q9: Intel Sky Lake

4-issue ALU pipeline



$$MinCPI = \frac{1}{8}$$

$$MinINTInst.CPI = \frac{1}{4}$$

$$MinMEMInst.CPI = \frac{1}{4}$$

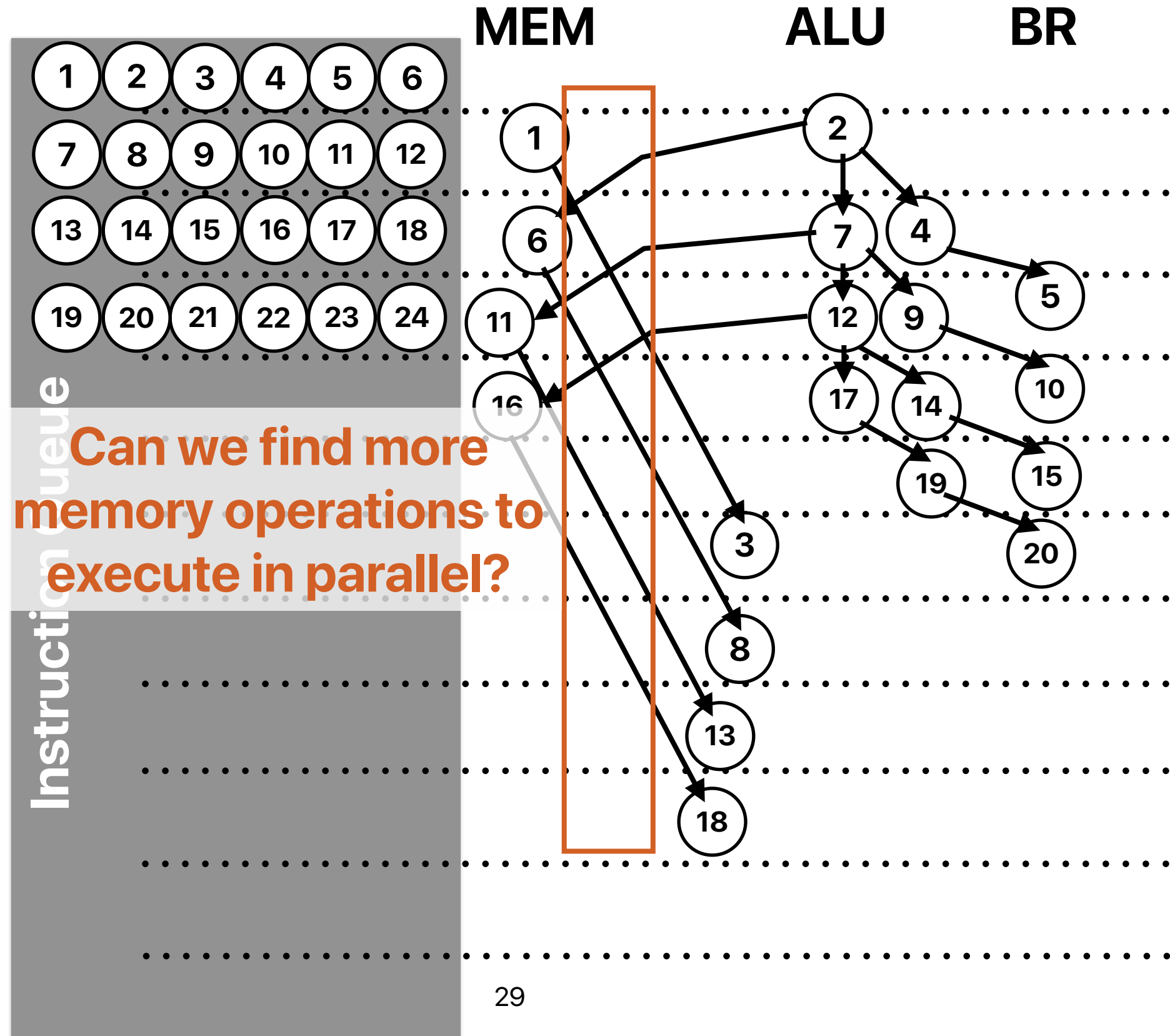
$$MinBRInst.CPI = \frac{1}{2}$$

4-issue memory pipeline

How can we improve performance?

```
for (i=0;i<size;i++)  
    sum += a[i];
```

```
① movl (%rdi), %ecx  
② addq $8, %rdi  
③ addq %rcx, %rax  
④ cmpq %rdx, %rdi  
⑤ jne .L3  
⑥ movl (%rdi), %ecx  
⑦ addq $8, %rdi  
⑧ addq %rcx, %rax  
⑨ cmpq %rdx, %rdi  
⑩ jne .L3  
⑪ movl (%rdi), %ecx  
⑫ addq $8, %rdi  
⑬ addq %rcx, %rax  
⑭ cmpq %rdx, %rdi  
⑮ jne .L3  
⑯ movl (%rdi), %ecx  
⑰ addq $8, %rdi  
⑱ addq %rcx, %rax  
⑲ cmpq %rdx, %rdi  
⑳ inc .L3
```




```

new_size = size >> 3
for (i=0;i<new_size*8;i++)
    sum += a[i];

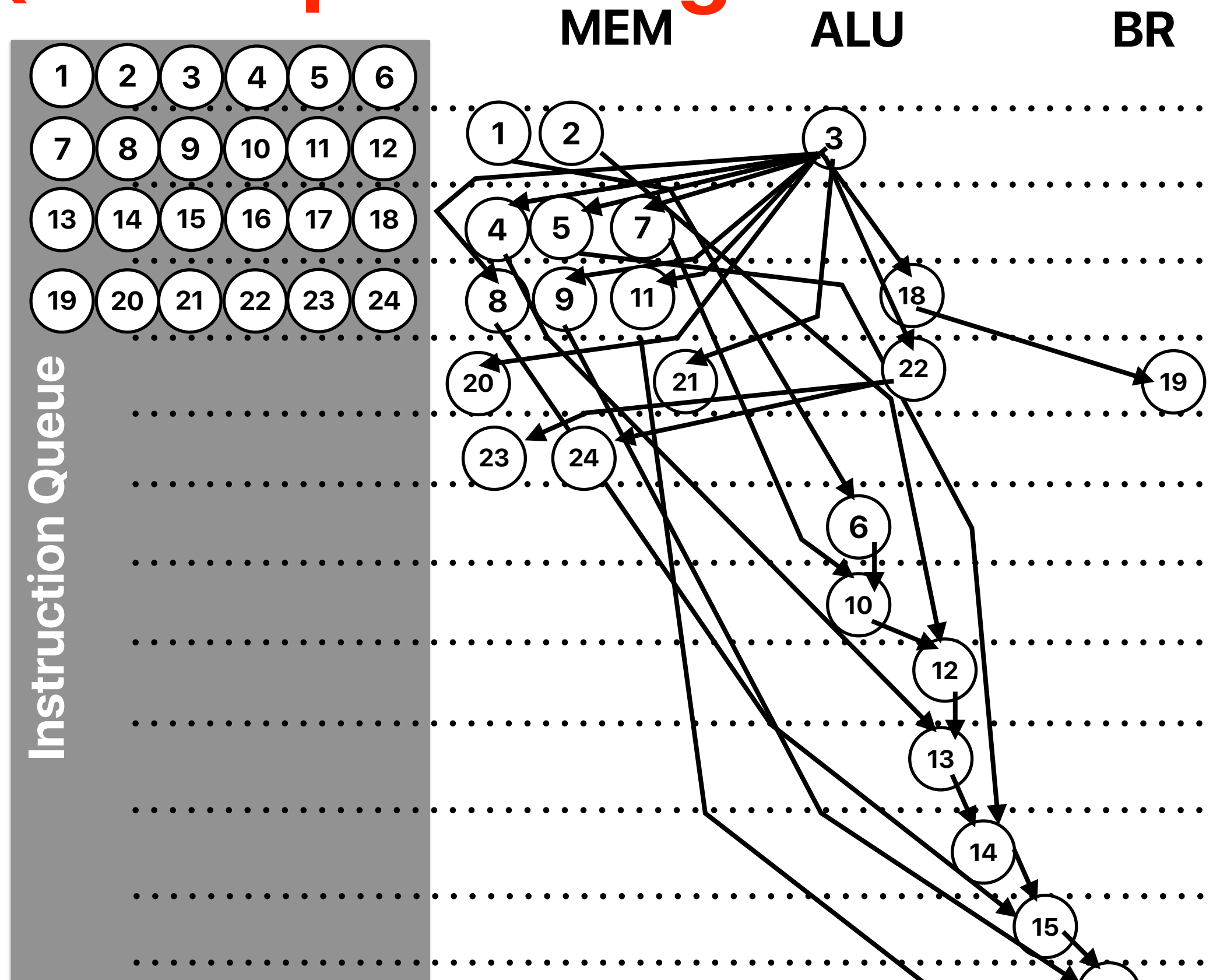
```

Q13, Q14: loop unrolling!

```

①  .L26: movslq    (%rdi), %r9
②      movslq    8(%rdi), %r10
③      addq     $32, %rdi
④      movslq   -20(%rdi), %r11
⑤      movslq   -16(%rdi), %rax
⑥      addq     %rcx, %r9
⑦      movslq   -28(%rdi), %rcx
⑧      movslq   -12(%rdi), %rsi
⑨      movslq   -8(%rdi), %r8
⑩      addq     %rcx, %r9
⑪      movslq   -4(%rdi), %rcx
⑫      addq     %r10, %r9
⑬      addq     %r11, %r9
⑭      addq     %rax, %r9
⑮      addq     %rsi, %r9
⑯      addq     %r8, %r9
⑰      addq     %r9, %rcx
⑱      cmpq    %rdx, %rdi
⑲      jne     .L26
⑳  .L26: movslq    (%rdi), %r9
㉑      movslq    8(%rdi), %r10
㉒      addq     $32, %rdi
㉓      movslq   -20(%rdi), %r11
㉔      movslq   -16(%rdi), %rax

```



Q16: How do we implement "switch"?

```
void teams_switch(uint64_t *pid, uint64_t *team, uint64_t total_number_of_students)
{
    uint64_t i;
    do {
        switch((pid[i] & 0x7)) {
            // V: taken if false
            case 0:
                team[i]=YELLOW;
                break;
            case 1:
                team[i]=RED;
                break;
            case 2:
                team[i]=BLUE;
                break;
            case 3:
                team[i]=ROCKET;
                break;
            case 4:
                team[i]=YELLOW;
                break;
            case 5:
                team[i]=RED;
                break;
            case 6:
                team[i]=BLUE;
                break;
            default:
                team[i]=ROCKET;
                break;
        }
        // 7: i < total_number_of_students means taken
    } while(++i < total_number_of_students);
    return;
}
```

```
_Z12teams_switchPmS_m:
.LFB3:
    endbr64
    pushq %rbp
    pushq %rbx
    movq %rdi, %r9
    movl $3, %ebp
    leaq .L12(%rip), %r8
    movl $2, %ebx
    movl $1, %r11d
    movl $0, %r10d
    jmp .L16
.L14:
    movq %r10, %rax
.L15:
    movq %rax, (%rsi,%rcx,8)
    addq $1, %rcx
    cmpq %rdx, %rcx
    jnb .L19
.L16:
    movq (%r9,%rcx,8), %rax
    andl $7, %eax
    cmpq $6, %rax
    ja .L10
    movslq (%r8,%rax,4), %rdi
    addq %r8, %rdi
    notrack jmp *%rdi
```

■ **cmpq** a, b sets flags based on b-a, but doesn't store

The "jump" table

```
.L12:
    .long .L14-.L12
    .long .L15-.L12
    .long .L15-.L12
    .long .L15-.L12
    .long .L14-.L12
    .long .L13-.L12
    .long .L11-.L12
    .text
.L13:
    movq %r11, %rax
    jmp .L15
.L11:
    movq %rbx, %rax
    jmp .L15
.L10:
    movq %rbp, %rax
    jmp .L15
.L19:
    popq %rbx
    popq %rbp
    ret
```

Retrieving the offset in the jump table

Calculating the address of the instruction

jump to the instruction where %rdi points to

Announcement

- Lab report 4 due this Sunday
- Programming assignment 3 is up and due 9/7/2025
 - 1.7x speedup is the minimum requirement
- Hall of fame of programming assignment 2
 - Aggressive loop unrolling
 - Matrix tiling

↕ Rank	↕ Submission Name	▼ speedup
1	<u>m</u>	1.262279562285401
2	<u>Harsh</u>	1.260194571775345
3	<u>keks</u>	1.23216490607218

Computer
Science &
Engineering

142L

つづく

